

```

import React, {
  forwardRef,
  useRef,
  useCallback,
  useState,
  useEffect,
} from "react";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface DateEditInnerIconConfig {
  icon: LucideIcon;
  className?: string;
}

export interface DateEditActionButtonConfig {
  icon: LucideIcon;
  onClick: (e: React.MouseEvent<HTMLButtonElement>, value: string) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface DateEditProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  dbFormat?: string;
  innerIcons?: DateEditInnerIconConfig[];
  actionButtons?: DateEditActionButtonConfig[];
  onChange?: (value: string, dbValue: string) => void;
}

const DEFAULT_DB_FORMAT = "YYYY-MM-DD";

const formatDateForDb = (dateString: string, dbFormat: string): string => {
  if (!dateString) return "";
  const date = new Date(dateString);
  if (isNaN(date.getTime())) return dateString;

  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");

```

```

    return dbFormat
      .replace("YYYY", String(year))
      .replace("MM", month)
      .replace("DD", day);
  };

const parseDateFromDb = (dbValue: string, dbFormat: string): string => {
  if (!dbValue) return "";
  const yearMatch = dbFormat.indexOf("YYYY");
  const monthMatch = dbFormat.indexOf("MM");
  const dayMatch = dbFormat.indexOf("DD");

  if (yearMatch === -1 || monthMatch === -1 || dayMatch === -1) return dbValue;

  const year = dbValue.substring(yearMatch, yearMatch + 4);
  const month = dbValue.substring(monthMatch, monthMatch + 2);
  const day = dbValue.substring(dayMatch, dayMatch + 2);

  return `${year}-${month}-${day}`;
};

export const DateEdit = forwardRef<HTMLInputElement, DateEditProps>(
  (
    {
      label,
      error,
      dbFormat = DEFAULT_DB_FORMAT,
      innerIcons = [],
      actionButtons = [],
      className,
      disabled,
      required,
      placeholder,
      value,
      defaultValue,
      onChange,
      onBlur,
      ...props
    },
    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const hasError = Boolean(error);
    const [displayValue, setDisplayValue] = useState<string>("");
    const [isFocused, setIsFocused] = useState(false);

    const handleRef = useCallback(
      (node: HTMLInputElement | null) => {
        inputRef.current = node;
        if (ref) {

```

```

        if (typeof ref === "function") {
            ref(node);
        } else {
            ref.current = node;
        }
    }
},
[ref],
);

useEffect(() => {
    const initialValue =
        value !== undefined
        ? String(value)
        : defaultValue !== undefined
        ? String(defaultValue)
        : "";
    if (initialValue) {
        const parsed = parseDateFromDb(initialValue, dbFormat);
        setDisplayValue(parsed);
    } else {
        setDisplayValue("");
    }
}, [value, defaultValue, dbFormat]);

const handleChange = useCallback(
    (e: React.ChangeEvent<HTMLInputElement>) => {
        const newValue = e.target.value;
        setDisplayValue(newValue);
        const dbValue = formatDateForDb(newValue, dbFormat);
        onChange?.(newValue, dbValue);
    },
    [dbFormat, onChange],
);

const handleBlur = useCallback(
    (e: React.FocusEvent<HTMLInputElement>) => {
        setIsFocused(false);
        onBlur?.(e);
    },
    [onBlur],
);

const handleFocus = useCallback(() => {
    setIsFocused(true);
}, []);

const limitedInnerIcons = innerIcons.slice(0, 4);
const limitedActionButtonButtons = actionButtons.slice(0, 4);

```

```

return (
  <div className={cn("w-full", className)}>
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
    <div className="flex items-center w-full gap-2">
      <div
        className={cn(
          "relative flex items-center flex-1 min-w-0",
          "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
          "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
          "rounded-surface",
          "text-text-main placeholder:text-muted-foreground/60",
          "focus-within:ring-2 focus-within:ring-amber-500/20",
          "focus-within:border-amber-500 focus-within:bg-amber-500/5",
          "disabled:opacity-50 disabled:pointer-events-none",
          "disabled:cursor-not-allowed",
          "transition-all duration-200 ease-out",
          hasError &&
            "border-destructive/50 focus-within:ring-destructive/50",
          "focus-within:border-destructive/50",
          disabled && "bg-muted/50",
          isFocused && "ring-2 ring-amber-500/20 border-amber-500",
        )}
      >
        <input
          ref={handleRef}
          type="date"
          className={cn(
            "flex-1 bg-transparent border-none outline-none",
            "px-4 py-2.5",
            "text-text-main placeholder:text-muted-foreground/60",
            "disabled:opacity-50 disabled:pointer-events-none",
            "disabled:cursor-not-allowed",
            "w-full min-w-0",
            limitedInnerIcons.length > 0 ? "pr-12" : "pr-4",
          )}
          disabled={disabled}

```

```

    required={required}
    placeholder={placeholder}
    value={displayValue}
    onChange={handleChange}
    onBlur={handleBlur}
    onFocus={handleFocus}
    {...props}
  />
  {limitedInnerIcons.length > 0 && (
    <div className="absolute right-3 flex items-center gap-1">
      {limitedInnerIcons.map((iconConfig, index) => (
        <span
          key={index}
          className={cn(
            "flex items-center justify-center",
            "w-5 h-5",
            "text-muted-foreground/60",
            iconConfig.className,
          )}
          aria-hidden="true"
        >
          <iconConfig.icon className="w-4 h-4" aria-hidden="true" />
        </span>
      ))}
    </div>
  )}
  </div>
  {limitedActionButtons.length > 0 && (
    <div className="flex items-center gap-1.5 shrink-0">
      {limitedActionButtons.map((action, index) => (
        <button
          key={index}
          type="button"
          className={cn(
            "relative flex items-center justify-center",
            "w-9 h-9 rounded-md",
            "text-muted-foreground/60 hover:text-foreground",
            "bg-transparent hover:bg-accent",
            "transition-transform duration-100",
            "active:scale-95",
            "focus:outline-none focus-visible:ring-2",
            "focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
            "disabled:opacity-50 disabled:pointer-events-none",
          )}
          aria-label={action.tooltipText}
          disabled={disabled || action.disabled}
          onClick={(e) => action.onClick(e, displayValue)}
        >
          <action.icon className="w-4 h-4" aria-hidden="true" />
        </button>
      ))}
    </div>
  )}

```

```

        )))
      </div>
    })
  </div>
  {error && (
    <p
      className={cn(
        "mt-1.5 text-sm",
        "text-destructive/90",
        "font-medium",
      )}
      role="alert"
    >
      {error}
    </p>
  )}
</div>
);
},
);

```

```

DateEdit.displayName = "DateEdit";

```